1. Linear search (employee name)
2. Recursive sum of array
3. Binary search in sorted array (product IDs)
4. Reverse a list (ints and strings) using recursion
5. Remove duplicates from a sorted list (in-place write index)
6. Index of highest score
7. Insert a new price into a sorted array (shift right)
8. First & last occurrence (modified binary searches)
9. Quick sort names alphabetically (manual quicksort)
10. Two numbers that sum to target (two-pointer)
11. Median of two sorted arrays (no full merge)
12. Sort students by marks (desc), then name (asc) bubble sort
13. Pascal's triangle up to n rows (iterative, fixed buffers)

1) Linear search (employee name) array + loop

cpp

```
#include <iostream>
using namespace std;

int main() {
    string employees[] = {"Alice", "Bob",
"Charlie", "Diana", "Erwin"};
    int n = 5;
    cout << "Enter employee name to search: ";
    string name;
    cin >> name;
    int index = -1;
    for (int i = 0; i < n; i++) {
        if (employees[i] == name) {
            index = i;
            break;
        }
    }
    if (index == -1) cout << "Not found\n";
    else cout << name << " found at index " <<
index << "\n";
    return 0;
}
```

2) Recursive sum of array

cpp

```
#include <iostream>
using namespace std;

int sumRec(int a[], int n) {
    if (n == 0) return 0;
    return a[n-1] + sumRec(a, n-1);
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int a[100];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> a[i];
    cout << "Sum = " << sumRec(a, n) << "\n";
    return 0;
}
```

3) Binary search in sorted array (product IDs)

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of product IDs (sorted):
";
    cin >> n;
    int ids[100];
    cout << "Enter sorted product IDs: ";
    for (int i = 0; i < n; i++) cin >> ids[i];
    cout << "Enter product ID to search: ";
    int x; cin >> x;
    int l = 0, r = n - 1, ans = -1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (ids[m] == x) {
            ans = m;
            break;
        }
        else if (ids[m] < x) l = m + 1;
        else r = m - 1;
    }
    if (ans == -1) cout << "Product not in
inventory\n";
    else cout << "Found at index " << ans << "\n";
    return 0;
}
```

4) Reverse a list (ints and strings) using recursion, no loops

cpp

```
#include <iostream>
using namespace std;

void printRevInt(int a[], int i) {
    if (i < 0) return;
    cout << a[i] << " ";
    printRevInt(a, i-1);
}

void printRevStr(string a[], int i) {
    if (i < 0) return;
    cout << a[i] << " ";
    printRevStr(a, i-1);
}

int main() {
    int n1, n2;
    cout << "Enter number of integers: ";
    cin >> n1;
    int nums[100];
    cout << "Enter integers: ";
    for (int i = 0; i < n1; i++) cin >> nums[i];
    cout << "Enter number of strings: ";
    cin >> n2;
    string names[100];
    cout << "Enter strings: ";
    for (int i = 0; i < n2; i++) cin >>
names[i];
    cout << "Reversed ints: ";
    printRevInt(nums, n1-1);
    cout << "\nReversed strings: ";
    printRevStr(names, n2-1);
    cout << "\n";
    return 0;
}
```

5) Remove duplicates from a sorted list (in-place write index)

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements (sorted): ";
    cin >> n;
    int a[100];
    cout << "Enter sorted elements: ";
    for (int i = 0; i < n; i++) cin >> a[i];
    int k = 0; // next write position
    for (int i = 0; i < n; i++) {
        if (k == 0 || a[i] != a[k-1]) {
            a[k] = a[i];
            k++;
        }
    }
    cout << "Cleaned list: ";
    for (int i = 0; i < k; i++) cout << a[i] << "
";
    cout << "\n";
    return 0;
}
```

6) Index of highest score

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of scores: ";
    cin >> n;
    int scores[100];
    cout << "Enter scores: ";
    for (int i = 0; i < n; i++) cin >>
scores[i];
    int bestIdx = 0;
    for (int i = 1; i < n; i++) {
        if (scores[i] > scores[bestIdx])
bestIdx = i;
    }
    cout << "Highest score = " <<
scores[bestIdx] << " at index " << bestIdx <<
"\n";
    return 0;
}
```

7) Insert a new price into a sorted array (shift right)

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of prices (sorted): ";
    cin >> n;
    double price[100];
    cout << "Enter sorted prices: ";
    for (int i = 0; i < n; i++) cin >> price[i];
    cout << "Enter new price: ";
    double p;
    cin >> p;
    int pos = n;
    for (int i = 0; i < n; i++) {
        if (p < price[i]) {
            pos = i;
            break;
        }
    }
    for (int i = n; i > pos; i--) price[i] =
price[i-1];
    price[pos] = p;
    n++;
    cout << "Updated prices: ";
    for (int i = 0; i < n; i++) cout << price[i]
<< " ";
    cout << "\n";
    return 0;
}
```

8) First & last occurrence (modified binary searches)

cpp

```
#include <iostream>
using namespace std;

int firstOcc(int a[], int n, int x) {
    int l = 0, r = n-1, ans = -1;
    while (l <= r) {
        int m = l + (r-l)/2;
        if (a[m] == x) {
            ans = m;
            r = m - 1;
        } else if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return ans;
}

int lastOcc(int a[], int n, int x) {
    int l = 0, r = n-1, ans = -1;
    while (l <= r) {
        int m = l + (r-l)/2;
        if (a[m] == x) {
            ans = m;
            l = m + 1;
        } else if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return ans;
}

int main() {
    int n, x;
    cout << "Enter number of elements
(sorted): ";
    cin >> n;
    int ids[100];
    cout << "Enter sorted elements: ";
    for (int i = 0; i < n; i++) cin >> ids[i];
    cout << "Enter ID to find: ";
    cin >> x;
    int f = firstOcc(ids, n, x);
    int g = lastOcc(ids, n, x);
    if (f == -1) cout << "ID not found\n";
    else cout << "First index = " << f << ",
Last index = " << g << "\n";
    return 0;
}
```

9) Quick sort names alphabetically (manual quicksort)

cpp

```cpp
#include <iostream>
using namespace std;

void swapStr(string &a, string &b) {
    string t = a; a = b; b = t;
}

int partitionArr(string a[], int low, int high) {
    string pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swapStr(a[i], a[j]);
        }
    }
    swapStr(a[i+1], a[high]);
    return i + 1;
}

void quickSort(string a[], int low, int high) {
    if (low < high) {
        int pi = partitionArr(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n;
    cout << "Enter number of names: ";
    cin >> n;
    string names[100];
    cout << "Enter names: ";
    for (int i = 0; i < n; i++) cin >> names[i];
    cout << "Before: ";
    for (int i = 0; i < n; i++) cout << names[i] << "
";
    cout << "\n";
    quickSort(names, 0, n-1);
    cout << "After: ";
    for (int i = 0; i < n; i++) cout << names[i] << "
";
    cout << "\n";
    return 0;
}
```

10) Two numbers that sum to target (two-pointer, no bool)

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements (sorted): ";
    cin >> n;
    int a[100];
    cout << "Enter sorted elements: ";
    for (int i = 0; i < n; i++) cin >> a[i];
    cout << "Enter target sum: ";
    int target;
    cin >> target;
    int i = 0, j = n - 1;
    while (i < j) {
        int s = a[i] + a[j];
        if (s == target) {
            cout << "Pair: " << a[i] << " + " <<
a[j] << " = " << target << "\n";
            return 0; // found; exit program
        } else if (s < target) {
            i++;
        } else {
            j--;
        }
    }
    cout << "No pair found\n";
    return 0;
}
```

11) Median of two sorted arrays (no full merge)

cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cout << "Enter size of first sorted array: ";
    cin >> n;
    int A[100];
    cout << "Enter first sorted array: ";
    for (int i = 0; i < n; i++) cin >> A[i];
    cout << "Enter size of second sorted array: ";
    cin >> m;
    int B[100];
    cout << "Enter second sorted array: ";
    for (int i = 0; i < m; i++) cin >> B[i];
    int total = n + m;
    int mid = total / 2;
    int i = 0, j = 0, count = 0;
    int prevVal = -1, curVal = -1;
    while (count <= mid) {
        prevVal = curVal;
        if (i < n && (j >= m || A[i] <= B[j])) {
            curVal = A[i];
            i++;
        } else {
            curVal = B[j];
            j++;
        }
        count++;
    }
    if (total % 2 == 1) cout << "Median = " <<
curVal << "\n";
    else cout << "Median = " << ((prevVal + curVal)
/ 2.0) << "\n";
    return 0;
}
```

12) Sort students by marks (desc), then name (asc) bubble sort

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    string name[100];
    int mark[100];
    cout << "Enter names:\n";
    for (int i = 0; i < n; i++) cin >> name[i];
    cout << "Enter marks:\n";
    for (int i = 0; i < n; i++) cin >> mark[i];
    for (int pass = 0; pass < n-1; pass++) {
        for (int i = 0; i < n-1-pass; i++) {
            int needSwap = 0;
            if (mark[i] < mark[i+1]) needSwap = 1;
            else if (mark[i] == mark[i+1] &&
name[i] > name[i+1]) needSwap = 1;
            if (needSwap) {
                int tmpM = mark[i];
                mark[i] = mark[i+1];
                mark[i+1] = tmpM;
                string tmpN = name[i];
                name[i] = name[i+1];
                name[i+1] = tmpN;
            }
        }
    }
    cout << "Sorted list:\n";
    for (int i = 0; i < n; i++) cout << name[i] <<
" " << mark[i] << "\n";
    return 0;
}
```

13) Pascal's triangle up to n rows (iterative, fixed buffers, no const/bool)

cpp

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n (<=30): ";
    if (!(cin >> n) || n < 1 || n > 30) {
        cout << "Invalid n\n";
        return 0;
    }
    int prev[31], curr[31];
    for (int i = 0; i < n; i++) {
        int len = i + 1;
        for (int j = 0; j < len; j++) {
            if (j == 0 || j == len - 1) curr[j] =
1;
            else curr[j] = prev[j - 1] + prev[j];
        }
        for (int j = 0; j < len; j++) cout <<
curr[j] << " ";
        cout << "\n";
        for (int j = 0; j < len; j++) prev[j] =
curr[j];
    }
    return 0;
}
```